

TECAN:

Typed ECAN for Cost-Accounted Attention Allocation in PLN and Algorithmic Chemistry

Ben Goertzel (with GPT-5.5-Pro and Claude Fable 5 High)

June 10, 2026

Abstract

Economic Attention Networks (ECAN) model attention allocation in OpenCog-like systems as an internal economy over atoms and links. ECAN’s central variables, Short-Term Importance (STI), Long-Term Importance (LTI), and the Attentional Focus (AF), have been useful abstractions for cognitive scheduling, associative memory, and credit assignment, including prior interactions with natural language processing and probabilistic reasoning. Classic ECAN, however, compresses many different scarcities into one or two scalar currencies. It cannot naturally distinguish, for example, the right to do a graph match from the right to do PLN deduction, abduction, truth-value revision, analogy, causal replay, context bridging, dequotation, or reproduction of a rule organism.

This paper proposes TECAN, Typed ECAN, as an upgrade of ECAN based on typed cost-accounted mortal computation. TECAN keeps ECAN’s economic and attentional intuition, but replaces scalar importance-spreading with a typed token metabolism over cognitive rewrites. Each atom, process, chamber, PLN inference frontier, or rule organism holds typed fuel; each candidate cognitive operation has a typed cost vector; reactions fire only when the required tokens are present; evaluators mint new typed fuel only after useful work; and starvation becomes the operational meaning of death for active cognitive processes. TECAN also absorbs information-geometric and fluidic ECAN variants: natural-gradient updates become cost-aware natural-gradient updates, and scalar attentional fluid dynamics becomes multi-species typed attentional fluid dynamics with shadow prices for token scarcity. We review ECAN and its variants, summarize Greg Meredith’s cost-accounted mortal-computation mathematics, define TECAN, give algorithms using the `algpseudocode` package, outline an incremental implementation path, explain how TECAN can guide PLN inference control, and then apply the same framework to algorithmic chemistry and semantic algorithmic chemistry. The result is a single resource semantics for focused inference, open-ended rule ecology, and controlled cognitive self-organization.

Contents

1	Introduction	3
2	ECAN and its Variants	4
2.1	Classic ECAN	4
2.2	ECAN in an integrative OpenCog setting	5
2.3	Information-geometric ECAN	5
2.4	Fluidic ECAN	6
2.5	What ECAN still lacks	7

3	Cost-Accounted Mortal Computation	7
3.1	Costed graph-structured lambda theories	7
3.2	Mortality and metabolism	7
3.3	Quote, dequote, genome, and soma	8
3.4	Why this matters for attention	8
4	TECAN: Typed ECAN	9
4.1	Design principle	9
4.2	State	9
4.3	Rules and costs	10
4.4	The TECAN gate	10
4.5	Evaluator-minted fuel	11
4.6	Old ECAN concepts inside TECAN	11
4.7	Cost-aware natural gradient	12
4.8	Typed attentional fluid dynamics	12
5	Core Algorithms	13
5.1	One TECAN epoch	13
5.2	Scarcity-aware causal scoring	13
5.3	Cost-aware natural-gradient gate learning	14
5.4	Typed fluidic TECAN transport	14
5.5	TECAN-driven mortal semantic chemistry	15
6	Incremental Implementation Path	16
6.1	Stage T0: instrumentation without behavior change	16
6.2	Stage T1: typed ledgers and hard gating	16
6.3	Stage T2: evaluator-minted fuel	17
6.4	Stage T3: causal replay and ACS detection	17
6.5	Stage T4: cost-aware natural gradient	17
6.6	Stage T5: typed fluidic transport	17
6.7	Stage T6: rule organisms and reproduction	17
6.8	Stage T7: portable worker IR and backend migration	17
7	TECAN and PLN	17
7.1	Why PLN is a central test case	17
7.2	PLN inference candidates as typed operations	18
7.3	Typed token families for PLN	18
7.4	Truth values and fuel are different ledgers	19
7.5	Scoring PLN inference steps	19
7.6	TECAN-guided PLN inference control	19
7.7	Examples of PLN typing	20
7.8	PLN versus semantic chemistry under TECAN	21
8	TECAN in Algorithmic Chemistry and Semantic Algorithmic Chemistry	21
8.1	Generic algorithmic chemistry	21
8.2	Semantic algorithmic chemistry	21
8.3	Mortal semantic ACSs	22
8.4	Typed semantic fuel	22

9	Initial Experiments and Metrics	23
9.1	Experiment 1: toy ECAN associative memory	23
9.2	Experiment 2: TECAN-guided PLN query answering	23
9.3	Experiment 3: toy semantic QA chamber	23
9.4	Experiment 4: proof-state chemistry	23
9.5	Experiment 5: software bug-pattern chemistry	24
9.6	Metrics	24
10	Risks and Guardrails	24
11	Discussion	25
12	Conclusion	25

1 Introduction

An integrative cognitive architecture cannot run every process on every atom all the time. It needs a way to decide what deserves immediate computation, what should remain in memory, what should be forgotten, and what should receive exploratory support despite uncertain payoff. ECAN answered this problem with an internal economy of attention. Atoms and links carry importance values; importance spreads over associative links; high-importance atoms enter an Attentional Focus; and wages, rents, taxes, welfare, and forgetting regulate the system’s short-term and long-term allocation of work.

That economic metaphor was productive because it made attention an active resource rather than a passive annotation. Yet the original currency was coarse. A single STI-like number was asked to summarize immediate salience, compute budget, memory retention pressure, inference-control priority, evidence route credit, and opportunity cost. In a small demonstration this may be good enough. In a system containing PLN, natural language processing, program learning, semantic graph chemistry, and rule-organism evolution, it is not. Such a system needs to know not just whether a region of memory is important, but what kind of scarce cognitive action that region is allowed to perform.

The distinction matters most when the same atom can participate in many different processes. A concept in the AF may be useful as a premise for a cheap inheritance deduction, as a starting point for an expensive abductive hypothesis, as a bridge into another context, as evidence to be revised against a conflicting source, or as a seed for semantic-chemistry rule mutation. Treating all of these as spending the same kind of attention obscures the real control problem. The system needs a typed budget, not merely more or less heat.

Meredith’s cost-accounted mortal-computation construction supplies a natural algebra for this refinement. A rewrite is not merely chosen by a scheduler; it is gated by the presence of matching tokens. Firing consumes tokens. Token splitters and joiners implement a metabolism. A process with no reachable matching fuel is dead by starvation. Native reflection then splits a quoted description, which is inert and copyable, from a dequoted running process, which is active and mortal. The result is a computational biology: genome, soma, metabolism, death, and life-history allocation arise from the same costed rewrite semantics.

This paper proposes to fuse these two ideas. TECAN, Typed ECAN, is ECAN made cost-accounted and typed. It is not a rejection of ECAN. It is a more explicit semantics for what ECAN was trying to approximate. TECAN treats attention as a typed permission system over cognitive rewrites. Importance is no longer only a scalar fluid that spreads over links. It is a

vector-valued, token-gated, causally priced metabolism that determines which atoms, PLN rule applications, semantic reactions, rule organisms, and algorithmic-chemistry loops may perform which classes of work.

The main thesis is therefore simple:

ECAN was a first-order economic metaphor for attention. TECAN makes the metaphor executable by replacing scalar importance spreading with typed, token-gated, causally priced cognitive metabolism.

The paper is organized as follows. Section 2 reviews classic ECAN and the information-geometric and fluidic variants that already suggest richer geometry and transport. Section 3 summarizes the relevant cost-accounting and mortal-computation mathematics. Section 4 defines TECAN. Section 5 gives core algorithms, formatted so that each algorithm stays inside its intended subsection. Section 6 gives an incremental implementation path. Section 7 explains how TECAN can work with PLN inference control. Section 8 explains the role of TECAN in algorithmic chemistry and semantic algorithmic chemistry. Section 9 lists initial experiments and metrics, and Section 12 concludes.

2 ECAN and its Variants

2.1 Classic ECAN

ECAN is easiest to understand as a book-keeping layer over a weighted cognitive graph. The graph may contain rich AtomSpace types that matter to other cognitive processes, but ECAN itself mostly treats atoms and links as locations where attention can be stored, moved, rewarded, charged, and forgotten. Certain links, especially HebbianLinks and InverseHebbianLinks, encode learned co-activation and anti-co-activation tendencies.

The two core attentional variables are short-term and long-term importance. We write them explicitly because the TECAN construction will later replace the underlying scalar liquidity by a typed ledger while still preserving these visible interfaces:

$$s_i(t) = \text{STI}_i(t), \tag{1}$$

$$\ell_i(t) = \text{LTI}_i(t). \tag{2}$$

STI is the fast variable that helps decide which atoms receive near-term processing. LTI is the slower variable that helps decide what should persist in memory. The AF is the immediate work set. In practice the AF is determined by rank, threshold, and cap rules; schematically it can be written as

$$\text{AF}(t) = \{i : \text{STI}_i(t) \geq \tau_{\text{AF}}\}. \tag{3}$$

The formula should not be read as the whole implementation. It is a compact way to say that ECAN converts a graded importance field into a bounded set of atoms that other cognitive processes may preferentially inspect.

The dynamics combine spreading, regulation, learning, and memory management. STI spreads over Hebbian and inverse-Hebbian links, often through a left-stochastic or approximately budget-preserving operator. Economic regulation adds wages for useful work, charges rent for remaining active, taxes over-concentration or other undesirable conditions, and supplies welfare to preserve exploration. Hebbian and inverse-Hebbian links are updated from co-activation statistics, especially co-membership in the AF. LTI modulates forgetting and persistence.

When ECAN is used as an associative memory, one may summarize the training dynamics as a nonlinear map. Let $x \in [0, 1]^L$ encode a binary or graded attentional pattern over L atoms. A training process with parameters θ can be viewed as

$$H : [0, 1]^L \rightarrow \mathbb{R}^M, \quad (4)$$

where M is the number of trainable connection parameters. This abstraction is useful because it lets us ask whether attention is being moved through the right geometry, not merely whether the update equations are locally plausible.

The virtue of ECAN is architectural simplicity: one coherent internal economy coordinates multiple cognitive processes. Its limitation is that the economy is mostly scalar. TECAN keeps the coordination role but refines the currency.

2.2 ECAN in an integrative OpenCog setting

ECAN was not meant to be a stand-alone attention toy. In OpenCog-style systems it is part of a cognitive-synergy architecture in which memory systems and cognitive processes share an AtomSpace. One illustrative line of work combined natural language processing with ECAN: parsed sentences stimulated WordNodes, ECAN spread STI through background knowledge, and related concepts could drift into the AF even when they were not directly present in the text. The same architecture explicitly treats PLN as the declarative reasoning process and ECAN as the attentional process, so the attention layer is already positioned to guide logical inference.

This history matters for TECAN because it shows both the promise and the weakness of scalar ECAN. Scalar attention can make related atoms available to PLN or NLP agents. But it does not say what sort of reasoning budget those atoms have earned. A word, concept, predicate, or implication may be attentionally hot, but the system still needs to decide whether to spend a cheap deduction token, an expensive abduction token, a revision token, or a cross-context bridge token. TECAN makes that distinction part of the attention semantics rather than an after-the-fact scheduler convention.

2.3 Information-geometric ECAN

The information-geometric variant of ECAN starts from the idea that an attention state is not just a vector in Euclidean space. A normalized attention state is a probability distribution over possible foci, and probability distributions have an intrinsic Fisher-Rao geometry. If the system is trying to move from a noisy or current attentional pattern toward a desired one, it should not necessarily follow the ordinary coordinate gradient.

Suppose a retrieval or training response is modeled as

$$y = H(x; \theta) + \xi, \quad (5)$$

where ξ is noise. If the conditional density decomposes as

$$p(y \mid x; \theta) = \prod_{i=1}^L r_i(y_i - H_i(x; \theta)), \quad (6)$$

then, under a Gaussian noise approximation with variance σ^2 , the Fisher information matrix has the convenient form

$$G(\theta) = \frac{1}{\sigma^2} E_x [\nabla H (\nabla H)^T]. \quad (7)$$

This matrix describes how changes in θ are actually seen by the distribution over outputs. The natural-gradient update therefore corrects the ordinary gradient by the inverse metric:

$$\theta_{t+1} = \theta_t - \eta_t G(\theta_t)^{-1} \nabla_{\theta} l(\theta_t), \quad (8)$$

where l is the loss. Adaptive natural-gradient learning avoids recomputing a full inverse metric from scratch. A typical online inverse-metric update is

$$\hat{G}_{t+1}^{-1} = (1 + \epsilon_t) \hat{G}_t^{-1} - \epsilon_t (\hat{G}_t^{-1} \nabla H) (\hat{G}_t^{-1} \nabla H)^T. \quad (9)$$

This variant can be read as geodesic ECAN. Attention should move through the Fisher-Rao geometry of attentional distributions rather than through a naive coordinate space. TECAN will keep this insight, but add typed token costs to the metric so that a statistically natural update must also be metabolically affordable.

2.4 Fluidic ECAN

A second ECAN variant interprets attention as a conserved fluid over a learned manifold M representing an active region of the AtomSpace. This picture is useful because it separates accounting from transport. ECAN remains the book-keeper, while a fluid step moves the same attention mass more coherently than isotropic diffusion.

Let $\rho(t, x)$ be attention density and $u(t, x)$ a velocity field. Between external stimulus and forgetting events, attention is moved by the continuity equation

$$\partial_t \rho + \nabla \cdot (\rho u) = 0, \quad (10)$$

$$\nabla \cdot u = 0. \quad (11)$$

The second equation says that the transport field is incompressible. It can move attention around, but it cannot secretly create or destroy the global budget.

The velocity field may itself be derived from an optimal-control problem. On the group $SG = \text{SDiff}(M)$ of volume-preserving diffeomorphisms, a value function W satisfies an HJB equation of the form

$$\partial_t W + \nu \Delta_{SG} W - \frac{1}{2} \|\nabla_{SG} W\|^2 - V = 0, \quad U^* = -\nabla_{SG} W. \quad (12)$$

Pushing the optimal policy U^* forward to M and projecting to divergence-free fields yields a Navier-Stokes-like transport equation

$$\partial_t u + (u \cdot \nabla) u = -\nabla p + \nu \Delta u + f_V, \quad \nabla \cdot u = 0. \quad (13)$$

In ECAN terms, p is a pressure or shadow price that prevents over-allocation, ν is an exploration-focus knob, and f_V encodes goals, queries, or cues. The usual interface can be summarized as

$$\text{STI} \mapsto \rho, \quad (14)$$

$$\text{AF} \mapsto \{x : \rho(x) > \tau\}, \quad (15)$$

$$\text{LTI} \mapsto \nu(x) \text{ and forgetting masks.} \quad (16)$$

TECAN generalizes this scalar fluid to a multi-species token fluid. There is no reason that deduction tokens, abduction tokens, context-bridge tokens, and mutation tokens should all move with exactly the same pressure field.

2.5 What ECAN still lacks

The variants above improve geometry and transport, but they still leave one basic question unspecified: what exactly is conserved, transported, and spent? A scalar STI value cannot distinguish a unit of matching budget from a unit of causal replay budget, a PLN revision budget, or a dequotation budget. TECAN begins by taking these distinctions literally.

3 Cost-Accounted Mortal Computation

3.1 Costed graph-structured lambda theories

The cost-accounted view starts from an ordinary rewriting substrate and adds metering. The substrate can be described as an interactive graph-structured lambda theory

$$G = (\Sigma, E, R), \quad (17)$$

where Σ is a signature, E is a set of structural equations, and R is a set of rewrite rules. An unmetered rewrite has the familiar form

$$\ell \rightarrow r. \quad (18)$$

The point of the cost construction is to prevent this rewrite from firing for free. The construction C extends G by adding token signatures, token stacks, signed or fuel-gated term wrappers, token-minting constructors, and a gated family of rules replacing each original rule.

If q names an original rule and $\kappa(q)$ is its cost, the metered form is schematically

$$\text{Tok}_{\kappa(q)} \mid \ell \rightarrow r. \quad (19)$$

The notation means that the redex fires only when the matching token or token bundle is present. Firing consumes that bundle. The scheduler is not merely asked to prefer one rewrite over another; the rewrite is operationally impossible without fuel.

For Turing-complete continued-interactive systems, the metered calculus CG can be translated back into the original system:

$$T : CG \rightarrow G. \quad (20)$$

Tokens become messages, signatures become channels, and signed terms become guards that must consume token messages before proceeding. The round trip

$$T \circ C : G \rightarrow G \quad (21)$$

identifies a distinguished sublanguage

$$\text{im}(T \circ C) \subseteq G \quad (22)$$

where mortality and metabolism are implemented using native constructs of G .

3.2 Mortality and metabolism

Once rewrites are token-gated, death no longer needs to be introduced as a separate primitive. A process is dead when every redex is fuel-gated and no matching token can ever reach any relevant gate. In words:

Death is permanent deadlock by fuel exhaustion.

Metabolism is the token economy that postpones death. Splitters turn large or compound tokens into smaller spendable units. Joiners assemble compound tokens from components. Gated rewrites couple computational work to expenditure. The boundary at which no token can be acquired is an absorbing boundary.

For TECAN we need the typed form of this story. Let $\mathcal{T}$ be a finite or computably generated token alphabet. A process P has a fuel vector

$$F_P \in \mathbb{N}^{\mathcal{T}}, \quad (23)$$

and a rule q has a typed cost vector

$$\kappa_q \in \mathbb{N}^{\mathcal{T}}. \quad (24)$$

The rule is fuel-enabled exactly when the process can pay every component of the bill:

$$F_P \geq \kappa_q. \quad (25)$$

The inequality is componentwise. A surplus of deduction tokens cannot pay for a missing abduction token unless the metabolism contains an explicit conversion route. When the rule fires, the ledger is debited by

$$F'_P = F_P - \kappa_q. \quad (26)$$

3.3 Quote, dequote, genome, and soma

If the substrate supports reflection, a quoted process $@P$ can be manipulated as inert code-as-data, while a dequoted process $*@P$ runs. This yields a computational germ/soma distinction:

Object	Role	Operational status
$@P$	germ-like description	inert, copyable, re-instantiable
$*@P$	soma-like process	active, metabolizing, mortal

The quoted object can be replicated without being interpreted. The dequoted object performs work and can starve. This distinction is essential for any TECAN account of rule organisms, because it lets the same rule set be copied, mutated, expressed, charged rent, and killed as an active process.

Regulated expression is token-gated dequotation:

$$\text{Tok}_{\kappa_{\text{dequote}}} \mid @P \rightarrow *@P. \quad (27)$$

Reproduction also becomes a ledgered operation, not a free meta-operation. A simple reproductive budget can be written as

$$B = c_{\text{maint}} + c_{\text{rep}}, \quad c_{\text{rep}} \approx n(c_{\text{mint}} + c_{\text{copy}} + e), \quad (28)$$

where B is realized income, c_{maint} is maintenance cost, n is the number of offspring, and e is offspring endowment. Different life-history strategies correspond to different allocations of the same finite budget.

3.4 Why this matters for attention

The cost-accounted construction says that a resource is not merely a scalar priority. It is a typed permission to perform a class of operation. This is precisely what scalar ECAN lacks. TECAN imports the typed fuel algebra into ECAN's attention economy.

4 TECAN: Typed ECAN

4.1 Design principle

TECAN keeps ECAN’s high-level purpose but changes the semantics of attention. In classic ECAN, STI is the immediately spendable attention currency. In TECAN, spendable attention is a typed fuel vector. A scalar importance value may still be useful for visualization, AF compatibility, or old process interfaces, but it is derived from a richer ledger.

The central design principle is:

Attention is typed rewrite-right. A cognitive operation may fire only when the appropriate typed attention tokens are available, and it survives only by producing value that causes evaluators to mint more tokens than it consumes.

The token alphabet should begin small enough to debug, but it must be expressive enough to distinguish qualitatively different cognitive costs. A useful initial alphabet is shown below; each item names a resource the system may want to budget separately.

Token type	Typical use
τ_{match}	graph matching and retrieval
τ_{deduce}	deduction or forward inference
τ_{induce}	inductive generalization
τ_{abduce}	abduction and hypothesis generation
$\tau_{\text{analogize}}$	analogy and mapping
τ_{revise}	truth-value revision and belief repair
τ_{cause}	causal intervention, replay, ablation
τ_{bridge}	cross-context transfer
$\tau_{\text{counterfactual}}$	counterfactual simulation
τ_{compress}	compression and macro formation
τ_{dequote}	expression of quoted rule genomes
τ_{mutate}	structural mutation of rule genomes
$\tau_{\text{reproduce}}$	copying and endowing offspring

4.2 State

A TECAN state needs to remember not only which atoms are hot, but also which processes own which fuel, which contexts have which shadow prices, and which past firings justify future credit. We therefore write the state as

$$S = (X, A, R, O, F, \Lambda, C, L, E), \tag{29}$$

where X is the atomspace or graph store, A is the set of atoms and links participating in attention allocation, R is the set of cognitive processes or rewrite rules, O is a population of active process bundles or rule organisms, F is the typed fuel ledger, Λ is the set of chamber-local shadow prices, C is the set of contexts or chambers, L is the event-log archive, and E is the evaluator and minting layer.

For an atom, process, or organism z , the underlying balance is a vector rather than a scalar:

$$F_z \in \mathbb{N}^T. \tag{30}$$

Old ECAN interfaces can still see a scalar attention value. That scalar should be read as a shadow-priced summary of typed liquidity and expected usefulness, for example

$$\text{STI}_z^{\text{vis}} = \psi(F_z, Q_z, \lambda_C), \quad (31)$$

where Q_z contains salience, expected utility, task relevance, and historical credit features. A simple concrete summary is

$$\text{STI}_z^{\text{vis}} = q_z + \sum_{\tau \in \mathcal{T}} w_{\tau, C} \log(1 + F_z(\tau)). \quad (32)$$

This preserves the old AF interface while making the underlying spendable liquidity typed.

4.3 Rules and costs

A TECAN operation is a possible act of cognition. It may be a PLN inference step, a semantic graph rewrite, a context bridge, a memory consolidation step, a natural-language interpretation step, a proof tactic, or a rule-organism mutation. What matters is that it has a precondition, an action, an owner, and a typed bill.

Formally, a candidate operation r has a precondition or graph pattern $P_r(X, C)$, an action or rewrite $X \mapsto r(X)$, an owner $o(r)$, a typed cost vector $\kappa_r \in \mathbb{N}^{\mathcal{T}}$, and a score model estimating plausibility, causal value, novelty, redundancy, risk, and future fuel production. The operation is fuel-enabled in context C exactly when

$$P_r(X, C) \wedge F_{o(r)} \geq \kappa_r. \quad (33)$$

When it fires, the graph, ledger, and log update together:

$$X' = r(X), \quad (34)$$

$$F'_{o(r)} = F_{o(r)} - \kappa_r, \quad (35)$$

$$L' = L \cdot \text{event}(o(r), r, \kappa_r, X, X'). \quad (36)$$

The log is not optional. Without provenance and replayable event records, TECAN cannot estimate which routes paid their rent.

4.4 The TECAN gate

The gate is the local decision rule that says whether an enabled operation is worth spending scarce typed fuel on now. It should reward plausibility, causal influence, novelty, compression, future fuel, and membership in useful rule organisms, while penalizing unsupportedness, contradiction, redundancy, and scarcity-weighted cost.

One representative gate is easier to read if we separate the probability from its logit score:

$$\text{Gate}_C(r, O, X) = \text{sigmoid}(Z_C(r, O, X)), \quad (37)$$

$$\begin{aligned} Z_C(r, O, X) = & a \logit(s_r) + b c_r + d \text{DoInf}_C(r) + e \text{Novelty}_C(r) + f \text{Compression}_C(r) \\ & + j \text{ACSBoost}_C(r) + q \text{FutureFuel}_C(r) - u \text{Unsupported}_C(r) - v \text{Contradiction}_C(r) \\ & - w \text{Redundancy}_C(r) - \lambda_C^T \kappa_r + h \log(1 + \text{Margin}(O, r)). \end{aligned} \quad (38)$$

Here s_r is a plausibility or truth-value strength, c_r is clarity, $\text{DoInf}_C(r)$ is an interventional influence estimate, and $\lambda_C^T \kappa_r$ is the shadow-priced token cost. The fuel margin is the tightest ratio of available to required fuel:

$$\text{Margin}(O, r) = \min_{\tau: \kappa_r(\tau) > 0} \frac{F_O(\tau)}{\kappa_r(\tau)}. \quad (39)$$

A rule that spends scarce tokens must earn a higher expected value to fire. A rule that consumes plentiful tokens may be allowed even when its payoff is more exploratory.

4.5 Evaluator-minted fuel

A crucial safety rule is that active somas should not mint their own survival fuel. Otherwise a loop can learn to counterfeit its own metabolic support. In TECAN, an evaluator reads the output, event log, provenance, and task context, then mints typed tokens. For output Y under query or task q , a representative minting score is

$$M_q(Y, L) = \alpha \text{Match}(Y, Y_q^*) + \beta \text{CausalClarity}(Y, L) + \gamma \text{Novelty}(Y) \quad (40)$$

$$+ \delta \text{Compression}(Y, L) + \eta \text{Transfer}(Y) - \mu \text{Unsupported}(Y, L) \quad (41)$$

$$- \nu \text{Contradiction}(Y) - \xi \text{Redundancy}(Y, L) - \zeta \text{Cost}(L). \quad (42)$$

The evaluator converts this score, possibly together with domain-specific features, into a typed mint vector

$$\Delta F = \text{Mint}_E(M_q, Y, L) \in \mathbb{N}^T. \quad (43)$$

A compact causal explanation may mint causal and compression tokens. A useful cross-context transfer may mint bridge tokens. A PLN proof step that advances a query may mint deduction or revision tokens. A rule organism that repeatedly solves a family of tasks may earn reproduction tokens.

4.6 Old ECAN concepts inside TECAN

The following table is not a one-to-one implementation spec. It is a map from old ECAN intuitions to TECAN semantics. The purpose is to keep what was useful in ECAN while making each quantity operationally auditable.

Classic ECAN	TECAN reinterpretation	Main benefit
STI	shadow-priced spendable fuel	keeps old AF interface
LTI	summary long-term credit and metabolic reliability	stabilizes retention
AF	enabled high-surplus work set	focuses on processes, not just atoms
HebbianLink	learned transport or conversion conduit	link must earn causal yield
Wage	evaluator-minted typed fuel	rewards useful work by type
Rent	maintenance cost	kills idle loops by starvation
Tax	crowding, redundancy, contradiction penalty	prevents runaway loops
Welfare	exploration endowment	supports uncertain high-upside work
Forgetting	starvation or archival	makes death operational
Rule learning	genome mutation/recombination	evolves rule organisms

4.7 Cost-aware natural gradient

Information geometry says that policy parameters should be updated in the intrinsic geometry of the distribution they induce. TECAN adds that such a move must also respect the geometry of token consumption. Let $\pi_\theta(r \mid X, C)$ be the stochastic policy induced by the TECAN gate, and let $J(\theta)$ be expected evaluator-minted value minus costs. The ordinary natural-gradient update is

$$\Delta\theta = \eta G_F(\theta)^{-1} \nabla_\theta J. \quad (44)$$

TECAN modifies the metric by adding token-consumption curvature terms:

$$\Delta\theta = \eta \left(G_F(\theta) + \sum_{\tau \in \mathcal{T}} \lambda_\tau C_\tau(\theta) + \epsilon I \right)^{-1} \nabla_\theta J. \quad (45)$$

Here G_F is the Fisher metric of the attentional policy and C_τ is a local metric or curvature term induced by changing expected consumption of token type τ . The same idea can be written as a constrained local optimization problem:

$$\max_{\Delta\theta} \quad \nabla J \cdot \Delta\theta, \quad (46)$$

$$\text{s.t.} \quad \Delta\theta^T G_F \Delta\theta \leq \epsilon_G, \quad (47)$$

$$\Delta\theta^T C_\tau \Delta\theta \leq B_\tau \quad \forall \tau \in \mathcal{T}. \quad (48)$$

The Lagrange multipliers of the token constraints become the shadow prices λ_τ . Fisher geometry says which policy changes are statistically natural; cost accounting says which natural changes are metabolically affordable.

4.8 Typed attentional fluid dynamics

Fluidic ECAN treated attention as one conserved fluid. TECAN treats it as a multi-species fluid. For each token type τ , let $\rho_\tau(t, x)$ be its density and $u_\tau(t, x)$ its transport field. The continuum approximation is

$$\partial_t \rho_\tau + \nabla \cdot (\rho_\tau u_\tau) = m_\tau(x, t) - c_\tau(x, t) + \sum_{\alpha, \beta} S_{\alpha\beta}^\tau j_{\alpha\beta}(x, t), \quad (49)$$

where m_τ is minting, c_τ is consumption, and the Sj terms implement splitter, joiner, and conversion flows. Between minting and consumption events, u_τ may be divergence-free. The pressure p_τ associated with token τ is the shadow price for scarcity or congestion of that token type.

On a graph, the corresponding discrete ledger is usually the practical starting point:

$$F_i^\tau(t+1) = F_i^\tau(t) + \sum_j J_{ji}^\tau(t) - \sum_j J_{ij}^\tau(t) + m_i^\tau(t) - c_i^\tau(t) + \text{Conv}_i^\tau(t). \quad (50)$$

This equation says that an atom or process gains a token type by inbound transport, minting, or conversion, and loses it by outbound transport or consumption. Scalar ECAN is recovered as the special case with one token type, no typed conversion, and a single visible STI ledger.

5 Core Algorithms

5.1 One TECAN epoch

A TECAN epoch is the smallest useful scheduling unit. It estimates current scarcity prices, collects affordable candidate operations, fires a bounded set of high-scoring operations, records their costs and effects, lets evaluators mint new fuel, and then applies rent, tax, welfare, AF updates, and link or organism updates. The point of the algorithm is not to replace all cognitive processes with one monolithic loop. It provides the accounting boundary inside which many processes can safely compete for typed rewrite-rights.

Algorithm 5.1 TECAN Epoch

Require: State $S = (X, A, R, O, F, \Lambda, C, L, E)$, chamber c , epoch budget B

Ensure: Updated state S'

```

1:  $\lambda_c \leftarrow \text{ESTIMATESHADOWPRICES}(F, c, L)$ 
2:  $Q \leftarrow \emptyset$ 
3: for all organisms or processes  $o \in O$  active in chamber  $c$  do
4:   for all candidate rules  $r \in R(o)$  do
5:     if  $\text{PRECONDITIONHOLDS}(r, X, c)$  and  $F_o \geq \kappa_r$  then
6:        $g \leftarrow \text{TECANSORE}(r, o, X, c, \lambda_c, F, L)$ 
7:       Insert  $(r, o, g)$  into priority queue  $Q$ 
8:     end if
9:   end for
10: end for
11:  $b \leftarrow 0$ 
12: while  $Q \neq \emptyset$  and  $b < B$  do
13:    $(r, o, g) \leftarrow \text{POPBEST}(Q)$ 
14:   if  $F_o \geq \kappa_r$  and  $\text{GATEACCEPTS}(g)$  then
15:      $X_{old} \leftarrow X$ 
16:      $X \leftarrow \text{APPLYREWRITE}(r, X, c)$ 
17:      $F_o \leftarrow F_o - \kappa_r$ 
18:      $L \leftarrow L \cdot \text{event}(o, r, \kappa_r, X_{old}, X)$ 
19:      $b \leftarrow b + \text{UNITCOST}(\kappa_r)$ 
20:   end if
21: end while
22:  $\Delta F \leftarrow \text{EVALUATORMINT}(E, X, L, c)$ 
23:  $F \leftarrow F + \Delta F$ 
24:  $F \leftarrow \text{APPLYRENTTAXWELFARE}(F, O, A, L, c)$ 
25:  $A \leftarrow \text{UPDATEAFANDVISIBLESTI}(A, F, \lambda_c)$ 
26:  $R, O \leftarrow \text{UPDATELINKSANDORGANISMS}(R, O, L, F, c)$ 
27: return  $(X, A, R, O, F, \Lambda, C, L, E)$ 

```

5.2 Scarcity-aware causal scoring

The epoch algorithm calls a scoring function rather than hard-coding one global priority rule. This makes it possible to start with a simple linear score and later learn the coefficients. The score below combines plausibility, causal influence, novelty, compression, ACS membership, expected future fuel, and fuel margin, while subtracting unsupportedness, contradiction risk, redundancy,

and shadow-priced cost. It is intentionally modular: any term can first be a cheap heuristic and later be replaced by a replay or ablation estimate.

Algorithm 5.2 TECAN Scarcity-Aware Causal Score

Require: Rule r , owner o , state X , chamber c , price vector λ_c , fuel F , log L

Ensure: Gate score $g \in [0, 1]$

- 1: $s \leftarrow \text{PLAUSIBILITYSTRENGTH}(r, X, c)$
 - 2: $d \leftarrow \text{DOINFLUENCEESTIMATE}(r, X, c, L)$
 - 3: $n \leftarrow \text{NOVELTY}(r, X, c, L)$
 - 4: $z \leftarrow \text{COMPRESSIONGAIN}(r, X, c, L)$
 - 5: $a \leftarrow \text{ACSBOOST}(r, o, L)$
 - 6: $q \leftarrow \text{EXPECTEDFUTUREFUEL}(r, o, X, c, L)$
 - 7: $u \leftarrow \text{UNSUPPORTEDNESS}(r, X, c, L)$
 - 8: $v \leftarrow \text{CONTRADICTIONRISK}(r, X, c, L)$
 - 9: $w \leftarrow \text{REDUNDANCY}(r, X, c, L)$
 - 10: $m \leftarrow \min_{\tau: \kappa_r(\tau) > 0} F_o(\tau) / \kappa_r(\tau)$
 - 11: $\text{cost} \leftarrow \lambda_c^T \kappa_r$
 - 12: $\text{zscore} \leftarrow a_1 \text{logit}(s) + a_2 d + a_3 n + a_4 z + a_5 a + a_6 q$
 - 13: $\text{zscore} \leftarrow \text{zscore} - b_1 u - b_2 v - b_3 w - \text{cost} + b_4 \log(1 + m)$
 - 14: **return** $\text{sigmoid}(\text{zscore})$
-

5.3 Cost-aware natural-gradient gate learning

The previous scoring rule can be learned from replay data. A standard policy update would move the gate parameters in the direction of expected reward. A natural-gradient update corrects this motion by the Fisher geometry of the gate policy. TECAN adds token-cost curvature so that the learned policy does not improve reward by creating hidden demand for scarce tokens. The algorithm below is deliberately written with estimators as black boxes, because the first implementation should use diagonal or low-rank approximations before attempting large dense inverses.

Algorithm 5.3 Cost-Aware Natural-Gradient Update for TECAN Gates

Require: Gate parameters θ , replay batch $\mathcal{B}$, prices λ , step size η

Ensure: Updated parameters θ'

- 1: $\widehat{\nabla J} \leftarrow \text{POLICYGRADIENTESTIMATE}(\theta, \mathcal{B})$
 - 2: $\widehat{G}_F \leftarrow \text{FISHERESTIMATE}(\theta, \mathcal{B})$
 - 3: $\widehat{C} \leftarrow 0$
 - 4: **for all** token types $\tau \in \mathcal{T}$ **do**
 - 5: $\widehat{C}_\tau \leftarrow \text{COSTCURVATUREESTIMATE}(\theta, \mathcal{B}, \tau)$
 - 6: $\widehat{C} \leftarrow \widehat{C} + \lambda_\tau \widehat{C}_\tau$
 - 7: **end for**
 - 8: $M \leftarrow \widehat{G}_F + \widehat{C} + \epsilon I$
 - 9: $\Delta\theta \leftarrow \eta M^{-1} \widehat{\nabla J}$
 - 10: **return** $\theta + \Delta\theta$
-

5.4 Typed fluidic TECAN transport

Fluidic transport is useful when a shard is large enough that individual symbolic diffusion over every edge is wasteful, but structured enough that a local dense representation makes sense. The

algorithm below treats each token type as a fluid species with its own velocity, pressure, viscosity, and forcing. It captures a shard, materializes dense arrays, runs advection-projection steps, applies mint/consume/convert terms, computes visible STI, and emits a patch log rather than mutating the global store opaquely.

Algorithm 5.4 Typed Fluidic TECAN Transport on a Shard

Require: Shard S_h , typed fuel densities ρ_τ , velocities u_τ , prices λ_τ , time step Δt

Ensure: Patch log for typed fuel and visible STI updates

```

1: CAPTURESHARD( $S_h$ )
2: MATERIALIZEARRAYS( $S_h, \rho, u, \lambda$ )
3: for all token types  $\tau \in \mathcal{T}$  do
4:    $f_\tau \leftarrow \text{GOALANDPRICEFORCING}(S_h, \tau, \lambda_\tau)$ 
5:    $u^* \leftarrow u_\tau + \Delta t[-(u_\tau \cdot \nabla)u_\tau + \nu_\tau \Delta u_\tau + f_\tau]$ 
6:    $p_\tau \leftarrow \text{SOLVEPOISSON}(\nabla \cdot u^* / \Delta t)$ 
7:    $u_\tau \leftarrow u^* - \Delta t \nabla p_\tau$ 
8:    $\rho_\tau \leftarrow \rho_\tau - \Delta t \nabla \cdot (\rho_\tau u_\tau)$ 
9:    $\rho_\tau \leftarrow \text{APPLYMINTCONSUMECONVERTTERMS}(\rho_\tau, S_h, \tau)$ 
10: end for
11: COMPUTEVISIBLESTIANDAF( $S_h, \rho, \lambda$ )
12: return EMITPATCHLOG( $S_h$ )

```

5.5 TECAN-driven mortal semantic chemistry

When TECAN is used inside a semantic chemistry, the scheduling epoch is only the first half of the story. After semantic rewrites fire, the system should inspect the event log for recurrent loops that close on themselves, matter causally, and earn more typed fuel than they spend. The algorithm below turns such loops into quoted genomes, supports reproduction when affordable, and kills or archives starved active somas.

Algorithm 5.5 TECAN Chamber for Mortal Semantic Chemistry

Require: Chamber c , semantic graph X_c , organism population O_c , evaluator E_c

Ensure: Updated chamber, organisms, and event logs

```
1: Run Algorithm 5.1 to fire affordable semantic rewrites
2:  $A_c \leftarrow \text{DETECTCANDIDATEACSS}(L_c, X_c, O_c)$ 
3: for all candidate ACS  $A \in A_c$  do
4:    $\text{surplus} \leftarrow \text{ESTIMATETOKENSURPLUS}(A, L_c, E_c)$ 
5:    $\text{impact} \leftarrow \text{ABLATIONIMPACT}(A, X_c, L_c, E_c)$ 
6:   if  $\text{surplus} > 0$  and  $\text{impact} > \tau_{\text{impact}}$  then
7:      $@R_A \leftarrow \text{REIFYASQUOTEDGENOME}(A)$ 
8:      $O_c \leftarrow O_c \cup \text{CREATEORPROMOTEORGANISM}(@R_A, A)$ 
9:   end if
10: end for
11: for all organisms  $o \in O_c$  do
12:   if  $\text{SHOULDREPRODUCE}(o, F_o, L_c)$  and  $F_o \geq \kappa_{\text{copy}} + \kappa_{\text{mut}} + e$  then
13:      $@R_{\text{child}} \leftarrow \text{MUTATEORRECOMBINE}(@R_o, O_c)$ 
14:      $F_o \leftarrow F_o - \kappa_{\text{copy}} - \kappa_{\text{mut}} - e$ 
15:      $O_c \leftarrow O_c \cup \{(@R_{\text{child}}, X_{\text{seed}}, e, c, L_{\text{new}})\}$ 
16:   end if
17:   if  $\text{STARVED}(o, F_o)$  then
18:      $O_c \leftarrow \text{ARCHIVEORKILL}(o, O_c)$ 
19:   end if
20: end for
21: return updated  $(X_c, O_c, L_c)$ 
```

6 Incremental Implementation Path

A useful TECAN implementation should not begin by replacing ECAN everywhere. The safest path is to preserve classic ECAN interfaces while adding typed accounting underneath them. Each stage below should be replayable before the next stage is added.

6.1 Stage T0: instrumentation without behavior change

The first stage only observes. Add event logs, process identifiers, and provisional cost annotations to an existing ECAN or inference-control system. Compute typed costs and hypothetical balances, but do not yet gate rule firing by them. This gives empirical traces from which token alphabets and costs can be calibrated.

Deliverables are event-log schemas for ECAN updates and cognitive process firings, cost-vector annotations for rules and processes, unchanged visible STI, and offline reports estimating which token types would have bottlenecked.

6.2 Stage T1: typed ledgers and hard gating

The second stage makes the ledger real. Introduce a small token alphabet, such as match, deduction, abduction, bridge, causal-replay, compression, and dequote. Assign every process a cost vector and every atom, process, or organism a balance. Permit firing only when the componentwise fuel constraint holds. The main goal is replayability and clear starvation diagnostics, not yet optimal performance.

6.3 Stage T2: evaluator-minted fuel

Once the system can starve, it needs a principled way to earn. Add task-specific evaluators and clamps. Outputs earn typed fuel according to answer quality, proof progress, test passing, causal clarity, novelty, compression, provenance, and transfer. Active somas should not directly mint their own reward fuel. Add rent, redundancy tax, contradiction tax, and unsupportedness tax.

6.4 Stage T3: causal replay and ACS detection

The third stage estimates which routes matter. Run paired replays in which rules, routes, or candidate ACSs are enabled, disabled, masked, or cooled. Estimate do-influence and ablation impact. Promote loops that have both positive causal impact and positive token surplus. Merge or starve loops that burn tokens without marginal yield.

6.5 Stage T4: cost-aware natural gradient

Only after the event log and evaluator are stable should gate parameters be learned aggressively. Train TECAN gate parameters with Algorithm 5.3. Begin with diagonal or low-rank approximations to the Fisher and cost-curvature matrices. Move to sparse or GPU-oriented methods once small cases are understood.

6.6 Stage T5: typed fluidic transport

Typed fluidic transport should be added as an optimization, not as a semantic shortcut. Implement typed token transport over dense shards using a MORK/ShardZipper-like pipeline: capture a shard, materialize structure-of-arrays buffers, run dense numeric kernels, emit a patch log, and splice the result back. Initially, use this only for token transport; keep rule semantics in the symbolic layer.

6.7 Stage T6: rule organisms and reproduction

Once surplus and ablation estimates are reliable, reify useful rule loops as quoted genomes. Implement token-gated dequotation, structural mutation, recombination, and offspring endowment. Maintain lineage logs and compare parent and child surplus under matched replay.

6.8 Stage T7: portable worker IR and backend migration

Durable semantics should live in portable facts rather than backend-private state. A TECAN worker should have explicit input facts, a chamber snapshot, a seed, a budget, and a backend tag; it should return an ordered event log, graph delta, token delta, and summary facts. This allows an initial PeTTa orchestration layer to be replaced later by a MeTTa-IL executor without changing the architecture.

7 TECAN and PLN

7.1 Why PLN is a central test case

Semantic chemistry is not the only natural application of TECAN. PLN is at least as important because ECAN and PLN already live together in the OpenCog architectural picture. PLN supplies uncertain logical inference over AtomSpace structures, while ECAN supplies attention allocation and credit assignment. Earlier OpenCog work explicitly treated PLN as a declarative reasoning

process and ECAN as the attentional process; related experimental work also pointed toward combining natural-language input, ECAN attention dynamics, and PLN reasoning over extracted logical knowledge.

This makes PLN a useful bridge between old ECAN and TECAN. In old ECAN, an atom with high STI becomes easier for PLN agents to inspect. In TECAN, the actual PLN inference frontier is costed. The system does not merely ask which atoms are hot. It asks which proposed PLN inference step, using which premises, under which substitution, in which context, deserves which typed rewrite-rights.

7.2 PLN inference candidates as typed operations

A PLN inference step is more structured than a generic attention update. It has a rule family, a tuple of premises, a substitution or binding, a target context, and a conclusion whose truth value must be computed or revised. TECAN therefore represents a candidate PLN act as

$$a = (q, \vec{p}, \theta, c, g), \quad (51)$$

where q is the PLN rule family, $\vec{p}$ is the premise tuple, θ is a variable binding or match substitution, c is a context, and g is the goal or query pressure the inference is meant to serve.

The typed cost of this act should reflect both the rule family and the concrete match. A useful first decomposition is

$$\kappa_a = \kappa_{\text{rule}}(q) + \kappa_{\text{match}}(\vec{p}, \theta) + \kappa_{\text{tv}}(q, \vec{p}, \theta) + \kappa_{\text{context}}(c) + \kappa_{\text{provenance}}(a). \quad (52)$$

The point of this decomposition is practical. A deduction rule over already-hot premises is cheap. An abductive rule with many possible hypotheses is expensive. A revision step that merges conflicting truth values costs revision and provenance tokens. A bridge from one context to another costs bridge tokens, and possibly analogy tokens if the transfer is not direct.

7.3 Typed token families for PLN

PLN’s richness gives TECAN somewhere useful to put type distinctions. A first PLN-focused alphabet can include the following token families.

Token family	Example PLN role	Typical risk if untyped
$\tau_{\text{pln_deduction}}$	forward implication chaining	cheap chains flood AF
$\tau_{\text{pln_induction}}$	generalization from cases	weak patterns overgeneralize
$\tau_{\text{pln_abduction}}$	hypothesis from observed effects	speculative hypotheses explode
$\tau_{\text{pln_analogy}}$	mapping between structures	analogy bridges drift
$\tau_{\text{pln_revision}}$	truth-value merge and repair	conflicts get ignored or overcounted
$\tau_{\text{pln_context}}$	context specialization or lifting	context leakage
$\tau_{\text{pln_grounding}}$	grounded evidence access	ungrounded loops inflate confidence
$\tau_{\text{pln_provenance}}$	support tracking and explanation	double counting hides

The table also indicates why scalar ECAN is too coarse. A system with plenty of cheap deduction capacity but little reliable provenance capacity should not keep building long chains whose support cannot be audited. A system rich in abduction tokens but poor in grounding tokens should produce hypotheses only under strong exploration constraints. A context bridge should pay a different bill from an in-context deduction.

7.4 Truth values and fuel are different ledgers

PLN already has an epistemic ledger: truth values. TECAN should not replace it. A truth value measures what the system believes and how well supported that belief is. Typed fuel measures what the system can afford to do. These are coupled but not identical.

For a simple PLN truth value, write

$$\text{TV}(A) = (s_A, c_A), \quad (53)$$

where s_A is strength and c_A is confidence. A PLN rule computes a conclusion truth value from premise truth values:

$$\text{TV}(B) = \text{PLNRule}_q(\text{TV}(P_1), \dots, \text{TV}(P_k)). \quad (54)$$

TECAN adds the operational condition that the rule application must be fuel-enabled:

$$\text{Enabled}(a, O) \iff \text{match}(\vec{p}, X, \theta) \wedge F_O \geq \kappa_a. \quad (55)$$

A high-confidence conclusion may still be a bad operation if it is redundant and costly. A low-confidence abduction may still be worth a welfare endowment if it has high expected future causal value. This distinction is essential: PLN truth values price epistemic support; TECAN prices metabolic opportunity.

7.5 Scoring PLN inference steps

A TECAN-PLN gate should score not just atoms but specific inference acts. The natural object of attention is therefore a quadruple or quintuple like $(q, \vec{p}, \theta, c, g)$. A representative PLN-specific score is

$$\text{Score}_{\text{PLN}}(a) = \alpha \text{GoalRelevance}(a, g) + \beta \text{ExpectedTVGain}(a) + \gamma \text{DoInf}(a) \quad (56)$$

$$+ \delta \text{NovelPremiseUse}(a) + \eta \text{ProofProgress}(a) - \mu \text{Circularity}(a) \quad (57)$$

$$- \nu \text{SupportOverlap}(a) - \xi \text{ContextLeakage}(a) - \lambda_c^T \kappa_a. \quad (58)$$

The score rewards a candidate for helping the active goal, improving expected truth values, making a causal difference to downstream answers, using premises in a novel but supported way, or shortening a proof. It penalizes circularity, redundant support, context leakage, and typed cost.

This is also where TECAN improves on simplistic ECAN-guided PLN. Old ECAN can say that a premise is in the AF. TECAN can say that a particular abductive use of that premise is too expensive right now, while a deductive use is affordable; or that a context bridge is allowed only if it spends bridge tokens and logs provenance.

7.6 TECAN-guided PLN inference control

The algorithm below is an explicit PLN counterpart to the generic TECAN epoch. It builds a frontier of possible PLN inference acts from the query, the AF, and the current context. It computes typed costs, rejects unaffordable acts, scores the rest, fires a bounded number, updates truth values and provenance, and then lets an evaluator mint fuel according to query progress and nonredundant support. The algorithm is deliberately epochal rather than fully asynchronous so that replay and ablation tests can reproduce the same inference frontier.

Algorithm 7.1 TECAN-Guided PLN Inference Epoch

Require: AtomSpace state X , query or goal g , context c , PLN rule set Q , fuel ledger F , log L , epoch budget B

Ensure: Updated AtomSpace, truth values, fuel ledger, and log

```
1:  $\mathcal{F} \leftarrow \text{BUILDPLNFRONTIER}(X, g, c, \text{AF}, Q)$ 
2:  $P \leftarrow \emptyset$ 
3: for all candidate acts  $a = (q, \vec{p}, \theta, c, g) \in \mathcal{F}$  do
4:    $\kappa_a \leftarrow \text{PLNCOSTVECTOR}(q, \vec{p}, \theta, c, L)$ 
5:    $o \leftarrow \text{OWNER}(a)$ 
6:   if  $F_o \geq \kappa_a$  then
7:      $score \leftarrow \text{PLNTECANSORE}(a, X, g, c, F, L)$ 
8:     Insert  $(a, score)$  into priority queue  $P$ 
9:   end if
10: end for
11:  $b \leftarrow 0$ 
12: while  $P \neq \emptyset$  and  $b < B$  do
13:    $(a, score) \leftarrow \text{POPBEST}(P)$ 
14:    $o \leftarrow \text{OWNER}(a)$ 
15:   if  $F_o \geq \kappa_a$  and  $\text{GATEACCEPTS}(score)$  then
16:      $(X, concl, tv) \leftarrow \text{FIREPLNRULE}(a, X)$ 
17:      $F_o \leftarrow F_o - \kappa_a$ 
18:      $L \leftarrow L \cdot \text{PLNEVENT}(a, concl, tv, \kappa_a)$ 
19:      $b \leftarrow b + \text{UNITCOST}(\kappa_a)$ 
20:   end if
21: end while
22:  $\Delta F \leftarrow \text{MINTFORPLNPROGRESS}(X, g, c, L)$ 
23:  $F \leftarrow \text{APPLYPLNRENTTAXWELFARE}(F, L, X, g, c) + \Delta F$ 
24:  $\text{AF} \leftarrow \text{UPDATEAF}(X, F, g, c)$ 
25: return  $(X, F, L, \text{AF})$ 
```

7.7 Examples of PLN typing

Consider a simple inheritance deduction. PLN may combine two premises to produce a transitive conclusion,

$$\text{Inheritance}(A, B), \quad \text{Inheritance}(B, C) \Rightarrow \text{Inheritance}(A, C). \quad (59)$$

The conclusion receives a truth value computed by PLN's truth-value formulas. In TECAN the same step consumes match, deduction, truth-value update, and provenance tokens. If the new conclusion already has strong independent support, the marginal yield may be low; the step may be rejected as redundant unless it improves confidence calibration or compresses a proof.

Abduction should be priced differently. Given an implication-like relation from H to E and evidence for E , PLN may hypothesize H . This can be valuable, but it can also explode into many speculative hypotheses. TECAN therefore makes abduction draw from a separate abduction token pool, and the evaluator mints more of that pool only when hypotheses later predict independent evidence, improve explanations, or guide useful action.

Context movement is another place where typing is decisive. A rule that is valid in a biomedical chamber, a fictional story-world chamber, or a specific dialogue context should not be lifted globally for free. TECAN can require context tokens, bridge tokens, analogy tokens, and provenance tokens

for such a move. The rule earns those tokens back only if replay shows that the transfer improves downstream answers without causing context leakage.

7.8 PLN versus semantic chemistry under TECAN

PLN and semantic chemistry should not be collapsed. PLN is a rule-governed uncertain logic with explicit truth-value algebra. Semantic chemistry is a more open-ended graph-rewrite ecology in which rule loops, motifs, and macro-patterns can become evolving organisms. TECAN is useful precisely because it can govern both without forcing them to be the same. In PLN, typed fuel controls the inference frontier. In semantic chemistry, typed fuel controls reaction firing, ACS survival, genome expression, mutation, and reproduction. The shared ledger allows the two to cooperate: a semantic ACS may earn deduction tokens by producing a reusable PLN macro-rule, and PLN may earn bridge or causal tokens by validating a semantic-chemistry hypothesis.

8 TECAN in Algorithmic Chemistry and Semantic Algorithmic Chemistry

8.1 Generic algorithmic chemistry

Algorithmic chemistry treats computational objects as molecules and rewrites as reactions. A reaction network may contain catalysts, autocatalytic sets, and higher-order objects such as rules that rewrite rules. The classical challenge is that open-ended chemistry can explode. Without explicit cost and mortality, fast cheap loops may dominate even when they do not do useful work.

TECAN supplies a typed metabolism for algorithmic chemistry. A reaction that would ordinarily be written as

$$r : M_1 + \cdots + M_k \rightarrow N_1 + \cdots + N_m \quad (60)$$

becomes a token-gated reaction

$$r : M_1 + \cdots + M_k + \text{Tok}_{\kappa_r} \rightarrow N_1 + \cdots + N_m. \quad (61)$$

The extra reactant is not decorative. It says the reaction fires only when the right typed fuel is available. Catalysis can now be made precise: a catalyst may lower a particular typed cost, improve fuel margin, or increase expected evaluator minting.

A loop survives only if its surplus is positive and if ablation shows downstream causal value:

$$E[\text{minted tokens} \mid A, C] - E[\text{spent tokens} \mid A, C] > 0. \quad (62)$$

The selected unit is therefore not just a molecule or a rule, but a costed autocatalytic process bundle.

8.2 Semantic algorithmic chemistry

Semantic algorithmic chemistry specializes algorithmic chemistry to meaning. The molecules are normalized semantic graphs. The reactions are context-sensitive semantic rewrites. The context may encode a domain, discourse frame, question type, proof state, fictional world, source regime, or reasoning style.

A plain semantic rewrite has the form

$$C \wedge G_1 \Rightarrow G_2. \quad (63)$$

The rule may carry a truth value, for example an intensional implication strength. One convenient schematic strength is

$$s_C(G_1, G_2) = \frac{\sum_{p \in \text{Pat}_C(G_1) \cap \text{Pat}_C(G_2)} w_C(p)}{\sum_{p \in \text{Pat}_C(G_1)} w_C(p) + \epsilon}. \quad (64)$$

This says that G_1 intensionally implies G_2 in context C when many of G_1 's characteristic context-relative properties also hold of G_2 .

TECAN upgrades the semantic rewrite to

$$C \wedge G_1 \wedge \text{HasFuel}(O, r) \Rightarrow G_2 \wedge \text{ConsumeFuel}(O, r) \wedge \text{EmitTrace}(O, r). \quad (65)$$

The added conjuncts are operational. They are the semantics of running the chemistry under mortality.

A useful semantic organism has the form

$$O = (@R, X, F, C, L), \quad (66)$$

where $@R$ is a quoted rule genome, X is a local semantic graph state, F is a typed fuel vector, C is a chamber, and L is an event log. The active soma $*@R$ fires graph rules only by spending typed fuel. The genome $@R$ can be copied, archived, mutated, recombined, and expressed.

8.3 Mortal semantic ACSs

The central object to discover is not a single good rule, but a self-maintaining rule ecology. A mortal semantic autocatalytic set is a pair

$$A = (M_A, R_A), \quad (67)$$

where M_A is a set of graph motifs and R_A is a set of semantic reactions. It should satisfy the following conditions.

1. **Closure.** Products of rules in R_A remain in M_A or return to M_A within bounded steps.
2. **Autocatalysis.** Rules in R_A are enabled or promoted by motifs produced by other rules in R_A .
3. **Metabolic surplus.** $E[\text{minted} \mid A, C] - E[\text{spent} \mid A, C] > 0$.
4. **Causal influence.** Ablating A changes reward, answer quality, proof progress, hypothesis utility, story coherence, prediction error, or future ACS formation.
5. **Heritability.** A can be reified as a quoted rule genome and copied, mutated, recombined, and re-expressed.

This is the key object for TECAN-driven semantic chemistry: a heritable, cost-accounted, self-maintaining inference-control organism.

8.4 Typed semantic fuel

Semantic chemistry needs its own domain-specialized token families. These can refine the general TECAN alphabet without changing the core ledger.

Token type	Example semantic role
$\tau_{\text{graph_match}}$	matching semantic graph patterns
$\tau_{\text{pln_deduction}}$	deductive inference inside semantic rules
$\tau_{\text{pln_abduction}}$	hypothesis generation
τ_{revision}	belief revision
τ_{analogy}	analogy construction
τ_{causal}	do-style replay and ablation
$\tau_{\text{context_bridge}}$	cross-chamber transfer
$\tau_{\text{paraphrase}}$	paraphrase collapse
$\tau_{\text{compression}}$	macro formation and explanation packing
$\tau_{\text{story_law}}$	fictional world-law inference
$\tau_{\text{biomed_mechanism}}$	mechanism hypothesis generation
$\tau_{\text{trial_design}}$	experiment or trial design

The alphabet should remain extensible. The point is not to choose the final ontology at the start, but to make distinct inference resources separable from the beginning.

9 Initial Experiments and Metrics

9.1 Experiment 1: toy ECAN associative memory

The first experiment should remain close to the older ECAN+ANGL benchmark style: noisy base patterns and retrieval accuracy. Compare classic ECAN, ECAN+ANGL, TECAN with typed match and retrieval tokens but no natural gradient, and TECAN with cost-aware natural gradient. Measure retrieval error, reward per token, starvation causes, and whether typed gating prevents useless high-draw retrieval paths.

9.2 Experiment 2: TECAN-guided PLN query answering

Use a small AtomSpace with ConceptNodes, PredicateNodes, EvaluationLinks, InheritanceLinks, ImplicationLinks, SimilarityLinks, context markers, and a set of PLN rules. Pose queries that can be solved by multiple routes: direct deduction, longer deduction chains, abduction followed by confirmation, analogy-assisted transfer, and revision of conflicting evidence. Compare scalar ECAN-guided PLN with TECAN-guided PLN.

The desired outcome is not simply more inference. It is better inference per typed token: fewer redundant chains, less circular support, clearer provenance, and more deliberate use of abduction and context bridging.

9.3 Experiment 3: toy semantic QA chamber

Use short texts parsed into semantic graphs and simple questions such as why, how, who, and what-caused. Expected ACSs include question-type detection, causal alignment, role completion, paraphrase collapse, answer-skeleton projection, and compact explanation packing. Reward compact causally grounded answers. Penalize unsupported bridges and redundant evidence paths.

9.4 Experiment 4: proof-state chemistry

Represent small theorem-proving tasks as proof-state graphs. Reward proof-state distance reduction, reusable lemma formation, proof compression, and successful replay. Penalize branch

explosion. Measure whether proof-strategy ACSs become heritable rule organisms.

9.5 Experiment 5: software bug-pattern chemistry

Represent code graphs, test failures, static warnings, candidate invariants, and patch schemas. External tests provide a strong evaluator. Reward minimal patches, transfer to related failures, and explanations that identify bug patterns. Penalize overfitting to tests.

9.6 Metrics

Task score alone is insufficient. TECAN should be measured as an ecology as well as a solver. Key metrics include:

- task reward per CPU millisecond;
- task reward per token type;
- number of active PLN inference frontiers and their token efficiency;
- number of active semantic ACSs;
- number of mortal ACSs with positive metabolic surplus;
- ablation impact of each ACS or PLN evidence route;
- lineage improvement from parent to child genomes;
- starvation rate and cause of death;
- redundancy tax paid by loops and duplicate routes;
- cross-context commutator defects;
- replay reproducibility under snapshot, seed, and budget;
- event-log equivalence across backends.

The central scientific question is:

Do useful, causally influential, token-efficient PLN inference routes and heritable semantic ACSs emerge, and do they improve when selected by typed token-gated metabolic pressure?

10 Risks and Guardrails

Several rules should be treated as implementation invariants.

1. Active somas must not mint their own reward fuel.
2. Loops should not be globally forbidden; they should pay rent and pass intervention tests.
3. A single scalar energy budget should not be used where typed tokens are needed for inference control.

4. PLN truth values must not be confused with TECAN fuel. Truth values are epistemic support; fuel is operational permission.
5. PeTTa-private predicates, if used, should not become architectural primitives; durable semantics should live in portable facts.
6. Concurrent workers should not mutate the same chamber hot pool without a deterministic reducer.
7. Novelty should not be rewarded without coherence, provenance, and task connection.
8. High-stakes domains should require stronger context guards, provenance, and evaluator constraints than creative exploratory domains.

11 Discussion

TECAN changes the interpretation of attention from “which atom is hot?” to “which operation has earned the right to spend which scarce resource?” This is important because modern cognitive systems are bottlenecked not only by total compute but by many qualitatively distinct kinds of reasoning opportunity. Deduction, abduction, analogy, causal replay, proof search, context bridging, truth-value revision, and genome mutation should not all draw from an indistinguishable pool.

This also clarifies the relationship between attention allocation and selection. In classic ECAN, high STI makes an atom more likely to be processed. In TECAN, typed fuel makes a process able to act. The difference is subtle but important: attention is not merely a signal to a scheduler, but a constitutive precondition of the computation. No token, no rewrite.

The PLN section illustrates why this is more than a semantic-chemistry idea. PLN already has rich atom types, inference rule families, and truth-value machinery. TECAN gives PLN a typed operational budget that can distinguish cheap deduction from expensive abduction, in-context inference from context bridging, and legitimate confidence improvement from circular double counting. Meanwhile semantic chemistry uses the same ledger to evolve rule organisms. This creates a possible bridge: PLN can validate semantic-chemistry hypotheses, and semantic chemistry can discover reusable macro-rules or inference motifs for PLN.

The information-geometric and fluidic variants are not discarded. They become higher-level approximations to the same costed economy. Natural gradient learns the gating policy in the intrinsic geometry of attentional distributions, while token costs add a metabolic metric. Fluid dynamics transports typed attention currencies through graph or manifold space, while pressure fields become typed shadow prices.

12 Conclusion

TECAN is a typed, cost-accounted version of ECAN. It preserves ECAN’s main insight—that cognitive attention should be allocated by an internal economy—but makes the economy explicit enough to support PLN inference control, algorithmic chemistry, and semantic algorithmic chemistry. Scalar STI becomes a visible summary of typed liquidity. Hebbian spreading becomes learned transport and conversion of scarce typed rewrite-rights. Wages become evaluator-minted typed fuel. Rent, tax, and welfare become maintenance, redundancy, contradiction, and exploration flows. Forgetting becomes starvation or archival. Rule learning becomes genome mutation and reproduction under explicit metabolic cost.

The resulting system can be summarized by one equation-like slogan:

TECAN = ECAN + typed mortal-computation metabolism + causal pricing + optional information-geometric and fluidic transport.

For PLN, the payoff is a typed inference frontier: the system can budget deduction, induction, abduction, analogy, revision, grounding, provenance, and context bridging separately while still using PLN truth values for epistemic support. For semantic algorithmic chemistry, the payoff is an ecology of rule organisms: quoted genomes are copied and expressed, active somas consume fuel, useful outputs earn fuel, and useless loops die. The same mechanism therefore supports focused logical reasoning and open-ended but controlled cognitive self-organization.

Acknowledgements

The author thanks Man Hin Leung for asking questions that prompted the TECAN concept. The paper also builds on Greg Meredith’s cost-accounting view of mortal computation and on prior ECAN, information-geometric ECAN, fluidic ECAN, PLN, algorithmic-chemistry, and semantic-chemistry work by their respective authors.

References

- [1] M. Ikle’ and B. Goertzel. Nonlinear-dynamical attention allocation via information geometry. Draft or conference manuscript, 2011.
- [2] M. Ikle’, J. Pitt, B. Goertzel, and G. Sellman. Economic Attention Networks: Associative memory and resource allocation for general intelligence. Proceedings of AGI, 2009.
- [3] M. B. Belachew, B. Goertzel, M. Ikle’, and D. Hanson. Shifting and drifting attention while reading: A case study of nonlinear-dynamical attention allocation in the OpenCog cognitive architecture. Draft manuscript, 2018.
- [4] B. Goertzel, M. Ikle’, I. Goertzel, and A. Heljakka. Probabilistic Logic Networks. Springer, 2008.
- [5] C. Harrigan, B. Goertzel, M. Ikle’, A. Belayneh, and G. Yu. Guiding probabilistic logical inference with nonlinear dynamical attention allocation. Proceedings of AGI-14, 2014.
- [6] S. Amari. Natural gradient works efficiently in learning. Neural Computation, 10:251–276, 1998.
- [7] H. Park, S. Amari, and K. Fukumizu. Adaptive natural gradient learning algorithms for various stochastic models. Neural Computation, 13:755–764, 2000.
- [8] L. G. Meredith. Mortal computation: Mortality, the genome, and life-history in cost-accounted GSLTs. Working note, May 31, 2026.
- [9] B. Goertzel. Mortal Semantic Chemistry: Token-gated semantic algorithmic chemistry and a PeTTa-to-MeTTa-IL implementation path. Draft manuscript, June 1, 2026.
- [10] B. Goertzel. Predictive Coding as Cost Optimization: Unifying the Meredith and Ororbis notions of mortal computation. Draft manuscript, June 2, 2026.

- [11] B. Goertzel. Going with the Flow – Attention as Incompressible Fluid Dynamics: From Bellman on $\text{SDiff}(M)$ to ECAN-on-NS, with a MORK/ShardZipper implementation sketch. Draft manuscript, October 6, 2025.
- [12] X.-D. Li and G. Liu. On the Navier-Stokes equations and the Hamilton-Jacobi-Bellman equation on the group of volume preserving diffeomorphisms. arXiv:2403.15997v3, 2025.
- [13] J. von Neumann. Theory of Self-Reproducing Automata. University of Illinois Press, 1966.